

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра Вычислительной техники

Курсовая работа
по дисциплине "Программирование"
Тема: «Разработка электронной картотеки»

Студент гр. 4311

Ласков С. В.

Преподаватель

Хахаев. И.А

Санкт-Петербург

2025

Введение

Цель работы

Разработка законченного программного решения задачи создания и сопровождения электронной картотеки мобильных телефонов. Требуется реализовать консольное приложение, которое позволяет пользователю работать с картотекой: добавлять, редактировать, удалять записи, выполнять сортировку и поиск по различным признакам, а также сохранять и загружать данные с диска.

Задание

Создать электронную картотеку, хранящуюся на диске, и программу, обеспечивающую взаимодействие с ней.

Программа должна выполнять следующие действия:

- занесение данных в электронную картотеку;
- внесение изменений (исключение, корректировка, добавление);
- поиск данных по различным признакам;
- сортировку по различным признакам;
- вывод результатов на экран и сохранение на диске.

Задача должна быть структурирована и отдельные части должны быть оформлены как функции.

Исходные данные должны вводиться с клавиатуры. В процессе обработки картотека должна храниться в памяти компьютера в виде списков и массивов структур, связанных указателями. Типы списков и структур выбираются исходя из предметной области(телефоны).

Постановка задачи и описание решения

Задача программы заключается в создании и сопровождении электронной картотеки, содержащей информацию о мобильных телефонах. Основные функции программы: добавление, редактирование, удаление карточек, поиск и сортировка по

различным параметрам (модель, бренд, объем RAM и т.д.), вывод данных на экран и сохранение в файл. Программа должна быть кроссплатформенной (Windows, Linux, macOS) и реализована на языке C с соблюдением стандарта C90.

Решение реализовано в виде консольного приложения. На этапе запуска программа загружает данные из CSV-файла и представляет их в оперативной памяти в виде односвязного списка. Пользователь взаимодействует с программой через меню. Все действия оформлены как отдельные функции, поддерживающие передачу данных через параметры и соблюдение принципов модульности.

Программа обеспечивает корректную работу с памятью (выделение и освобождение), проверяет ошибки при вводе, чтении и записи файлов. Данные о каждом телефоне представлены в виде структуры, содержащей основные характеристики: модель, бренд, RAM, объем памяти, цена, вес. Обработка осуществляется через динамические структуры данных — списки.

Программа поддерживает как вывод результатов на экран, так и их сохранение на диск. Пользователь может многократно выполнять действия, пока не выберет выход из программы.

Программа должна:

Добавлять карточки (данные вводятся пользователем);

Редактировать карточки;

Удалять карточки;

Печать всех карточек;

Поиск карточек по параметру (модель, бренд, RAM, и т.д.);

Сортировать карточки по параметру;

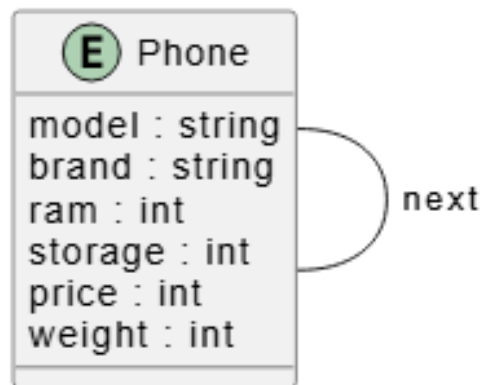
Сохранять и загружать данные из файла;

Выход из программы.

Архитектура данных Основная сущность — карточка телефона. Карточки связаны в односвязный список. Данные загружаются из

CSV-файла и представлены в оперативной памяти в виде списка структур.

Структура данных



Описание структур

Структура Brand

```
typedef struct Brand {
    int id;           // Уникальный идентификатор производителя
    char *name;       // Название бренда (выделяется динамически)
    struct Brand *prev; // Указатель на предыдущий бренд (для двусвязного списка)
    struct Brand *next; // Указатель на следующий бренд
} Brand;
```

Назначение:

Хранит информацию об уникальном производителе, используется для построения двусвязного списка брендов. Каждое название бренда встречается ровно один раз.

Структура Phone

```
typedef struct Phone {
    char model[MAX_FIELD_LENGTH]; // Название модели телефона
    Brand *brand;                 // Указатель на структуру Brand (производителя этого телефона)
    int ram;                      // Объем RAM (ГБ)
    int storage;                  // Объем хранения (ГБ)
    int price;                    // Цена
    int weight;                   // Вес (граммы)
    struct Phone *next;           // Указатель на следующий телефон в списке
};
```

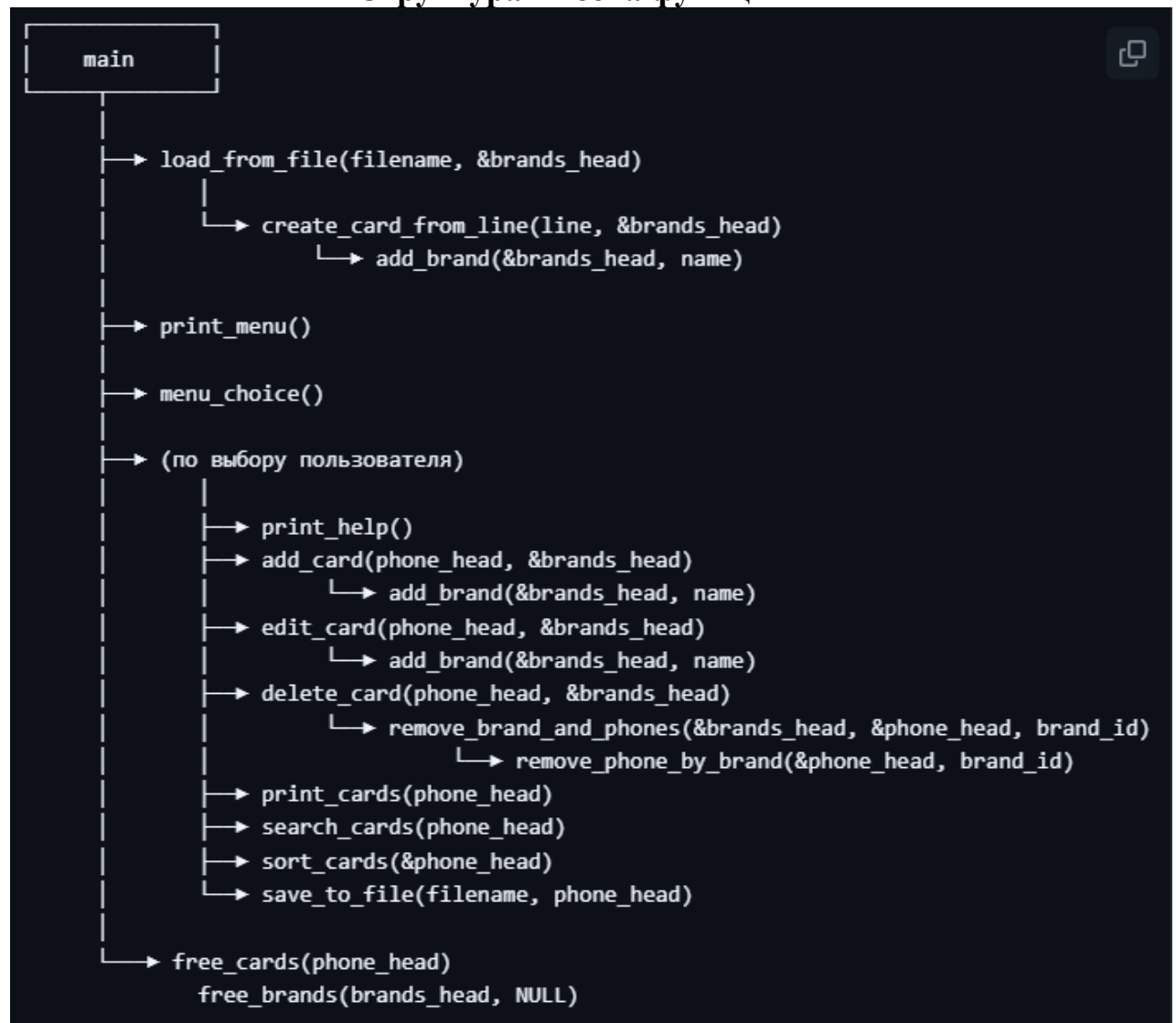
```
} Phone;
```

Назначение:

Описывает одну карточку объекта предметной области (телефон).

Содержит все параметры, а также указатель на бренд.

Структура вызова функций



Описание функций

void print_menu(): печатает главное меню

void print_help(): справка по программе

void pause_console(): пауза до нажатия Enter

void clear_screen(): очистка экрана (кроссплатформенно)

Phone* load_from_file(const char* filename): загрузка данных из CSV-файла

int save_to_file(const char* filename, Phone* head): сохранение списка в файл

Phone* add_card(Phone* head): добавление новой карточки в список

void print_cards(Phone* head): печать всех карточек на экран

Phone* edit_card(Phone* head): редактирование карточки по индексу

Phone* delete_card(Phone* head): удаление карточки из списка

void search_cards(Phone* head): поиск по параметру (строка или число)

void sort_cards(Phone** head): сортировка списка по выбранному параметру

void free_cards(Phone* head): освобождение динамически выделенной памяти

int get_int(const char* prompt, int min, int max): ввод целого числа с проверкой

void flush_input(void): очистка ввода (для обработки ошибок)

int menu_choice(int min, int max): выбор пункта меню

Phone* create_card_from_line(const char* line): создание структуры из строки CSV

void print_card(const Phone* card, int idx): печать одной карточки

Описание переменных

Функция: main

Название переменной	Тип	Назначение
head	Phone*	Указатель на начало списка карточек
running	int	Флаг продолжения выполнения программы
choice	int	Номер выбранного пользователем действия
filename	char[]	Имя CSV-файла для хранения данных

Функция: add_card

Название переменной	Тип	Назначение
card	Phone*	Указатель на новую добавляемую карточку
tail	Phone*	Указатель на последний элемент в списке
buf	char[MAX_FIELD_LENGTH]	Буфер для хранения ввода строки с клавиатуры

Функция: edit_card

Название переменной	Тип	Назначение
idx	int	Индекс редактируемой карточки
i	int	Счетчик цикла
p	Phone*	Указатель на текущую карточку
buf	char[MAX_FIELD_LENGTH]	Буфер для пользовательского ввода

Функция: delete_card

Название переменной	Тип	Назначение
idx	int	Индекс удаляемой карточки
i	int	Счетчик цикла
p	Phone*	Указатель на текущую карточку
prev	Phone*	Указатель на предыдущую карточку

Функция: search_cards

Название переменной	Тип	Назначение
param	int	Параметр поиска (по какому полю)
buf	char[MAX_FIELD_LENGTH]	Буфер для текстового поиска
val	int	Значение числового параметра поиска
p	Phone*	Указатель на текущую карточку
found	int	Счетчик найденных совпадений

Функция: save_to_file

Название переменной	Тип	Назначение
f	FILE*	Указатель на открытый файл для записи
p	Phone*	Текущая карточка для записи в файл

Функция: `sort_cards`

Название переменной	Тип	Назначение
<code>param</code>	<code>int</code>	Параметр сортировки
<code>swapped</code>	<code>int</code>	Флаг наличия обмена в итерации сортировки
<code>i</code>	<code>int</code>	Счетчик итераций
<code>p</code>	<code>Phone*</code>	Указатель на текущую карточку
<code>prev</code>	<code>Phone*</code>	Указатель на предыдущую карточку
<code>temp</code>	<code>Phone*</code>	Вспомогательная переменная для перестановки
<code>lptr</code>	<code>Phone*</code>	Ограничитель прохода по списку

Функция: `create_card_from_line`

Название переменной	Тип	Назначение
<code>card</code>	<code>Phone*</code>	Новая карточка
<code>buf</code>	<code>char[MAX_LINE_LENGTH]</code>	Буфер строки
<code>token</code>	<code>char*</code>	Текущий токен (поле)
<code>field</code>	<code>int</code>	Индекс поля

Функция: `load_from_file`

Название переменной	Тип	Назначение
<code>f</code>	<code>FILE*</code>	Указатель на открытый файл
<code>line</code>	<code>char[MAX_LINE_LENGTH]</code>	Буфер для строки из файла
<code>head</code>	<code>Phone*</code>	Указатель на начало списка
<code>tail</code>	<code>Phone*</code>	Указатель на конец списка
<code>card</code>	<code>Phone*</code>	Временная карточка, считанная из строки

Функция: get_int

Название переменной	Тип	Назначение
num	int	Введенное число
buf	char[MAX_LINE_LENGTH]	Буфер ввода
endptr	char*	Указатель на символ после числа
valid	int	Флаг корректности ввода

Функция: menu_choice

Название переменной	Тип	Назначение
num	int	Возвращаемое значение выбора меню

Функция: print_card

Название переменной	Тип	Назначение
card	const Phone*	Указатель на карточку
idx	int	Индекс карточки

Функция: print_cards

Название переменной	Тип	Назначение
<code>p</code>	Phone*	Текущий элемент списка при печати карточек
<code>idx</code>	int	Индекс выводимой карточки

Функция: free_cards

Название переменной	Тип	Назначение
<code>p</code>	Phone*	Временный указатель для освобождения памяти

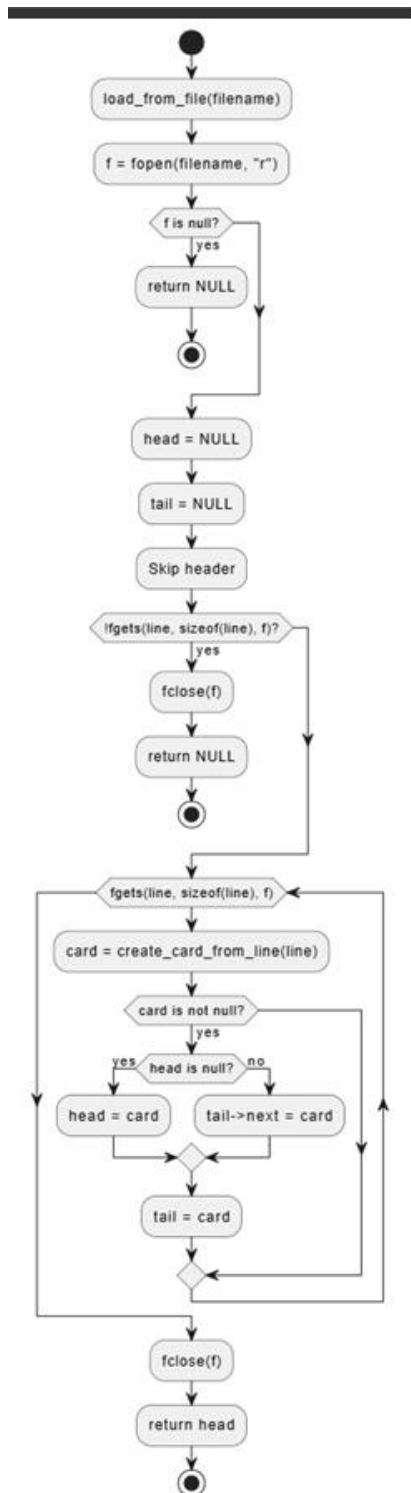
Функция: flush_input

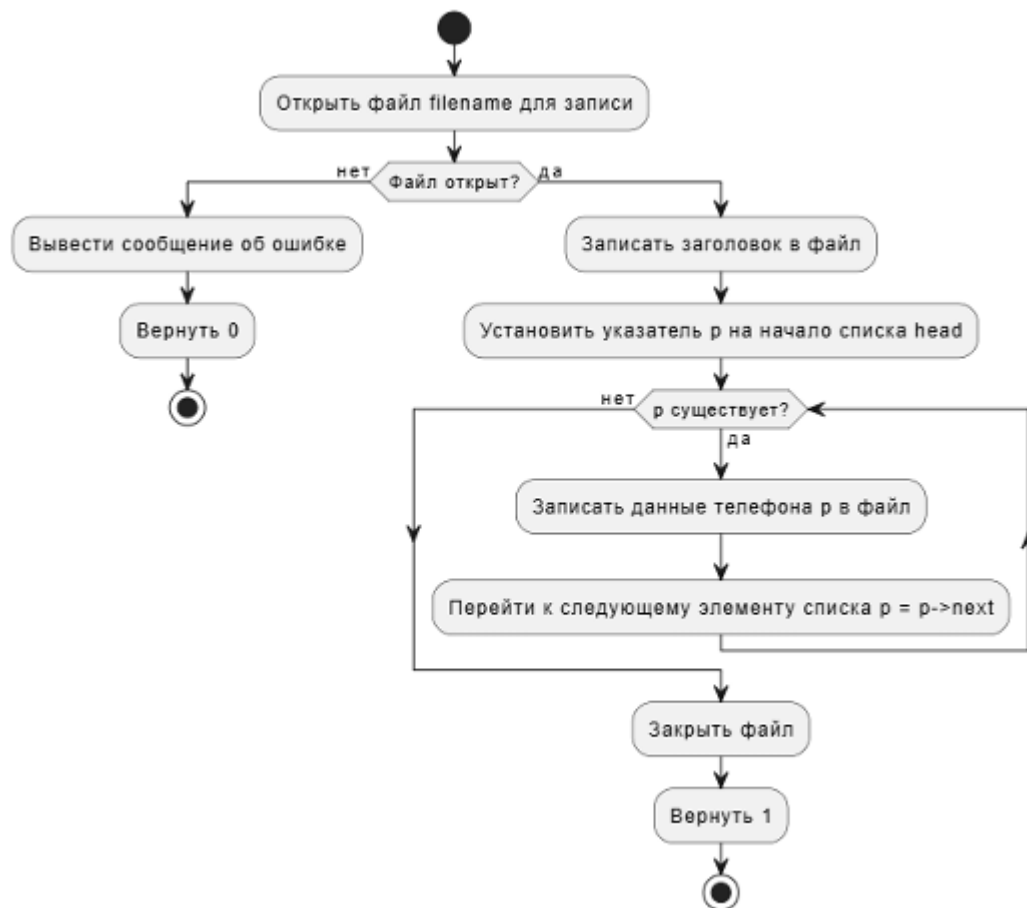
Название переменной	Тип	Назначение
c	int	Переменная для считывания символов из буфера ввода

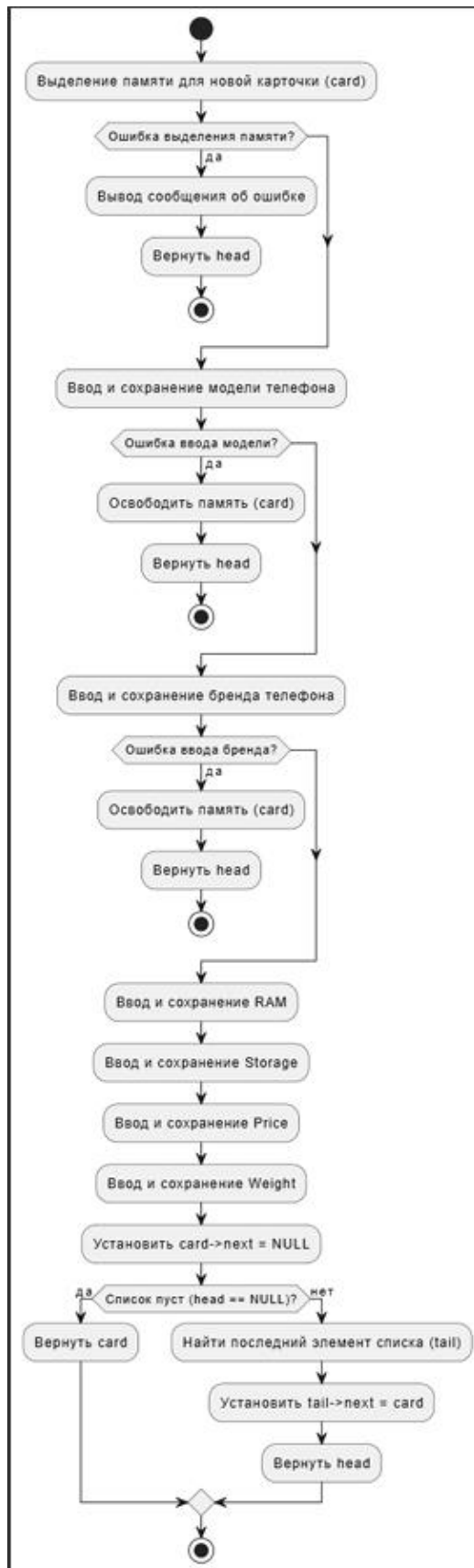
Функция: print_card

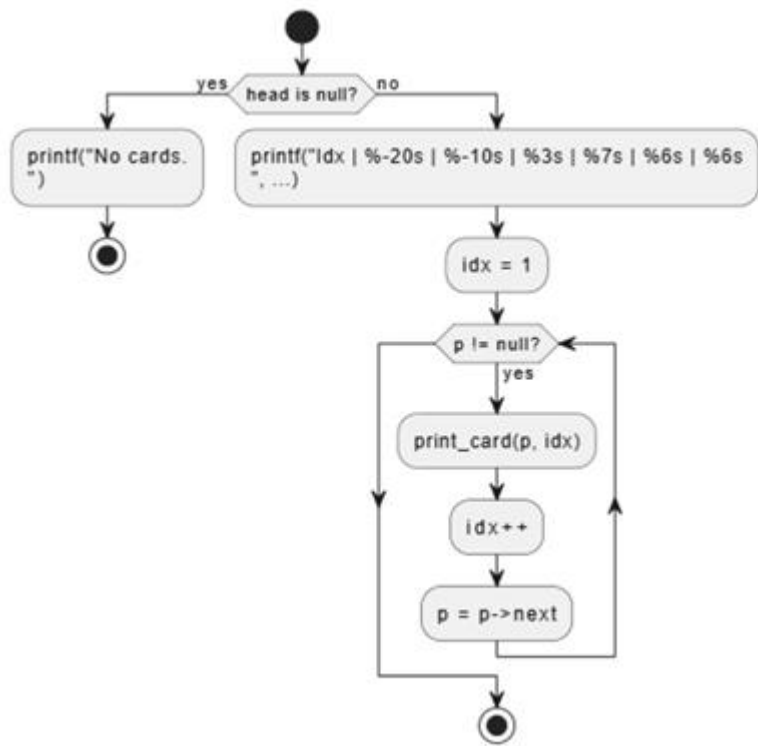
Название переменной	Тип	Назначение
card	const Phone*	Карточка для вывода
idx	int	Индекс карточки

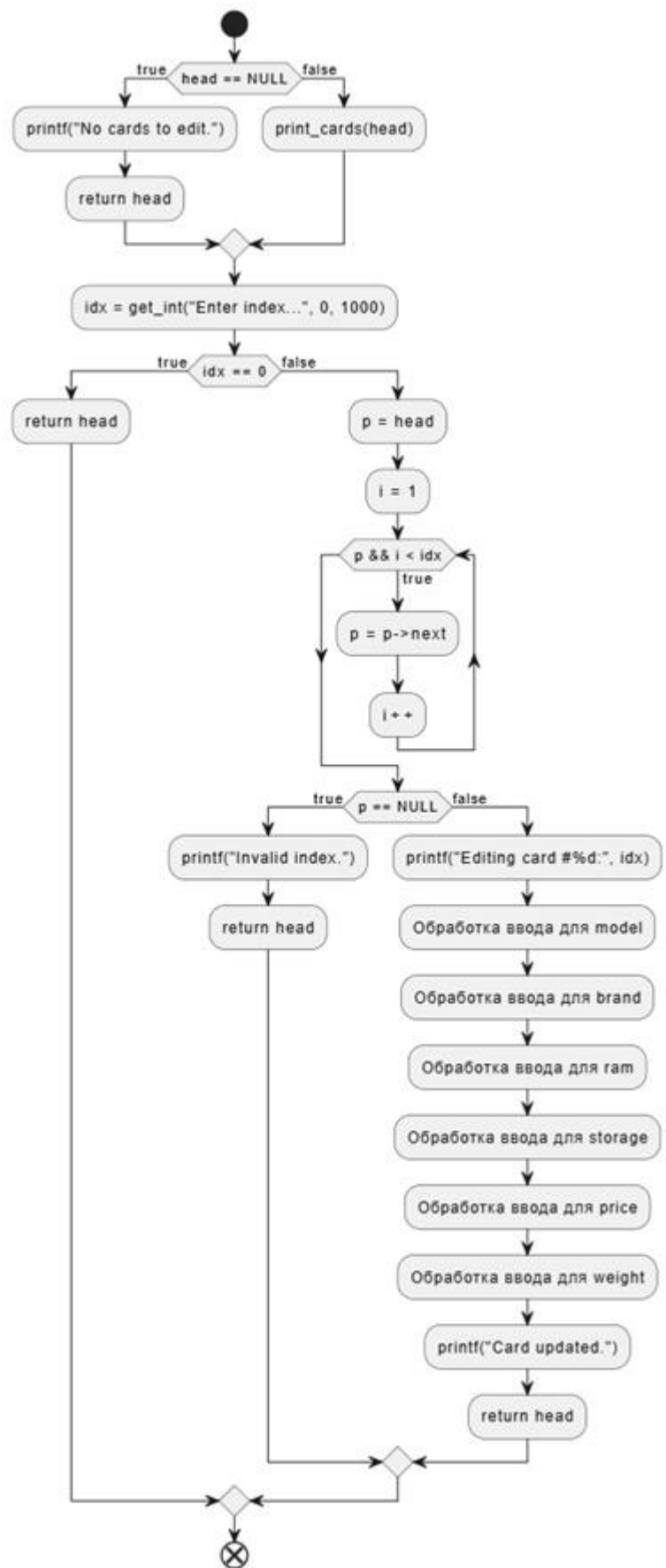
Схема алгоритма

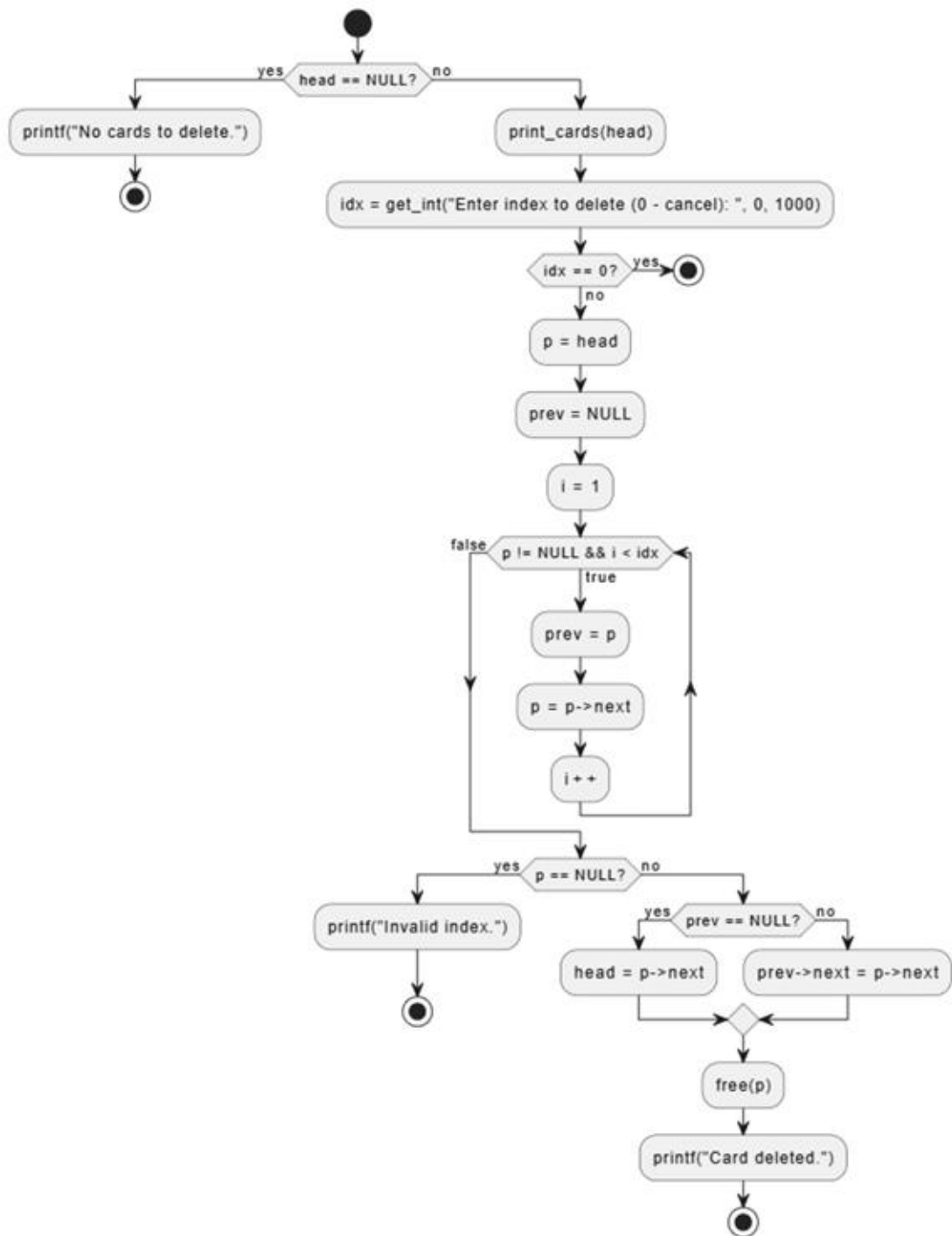




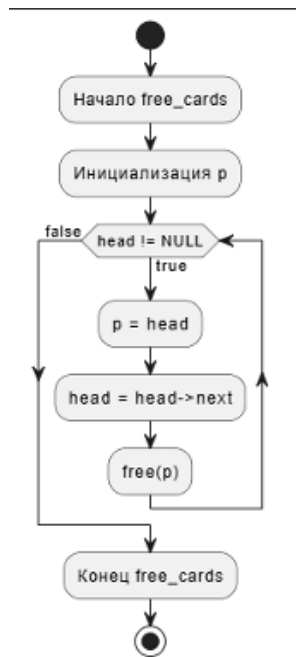
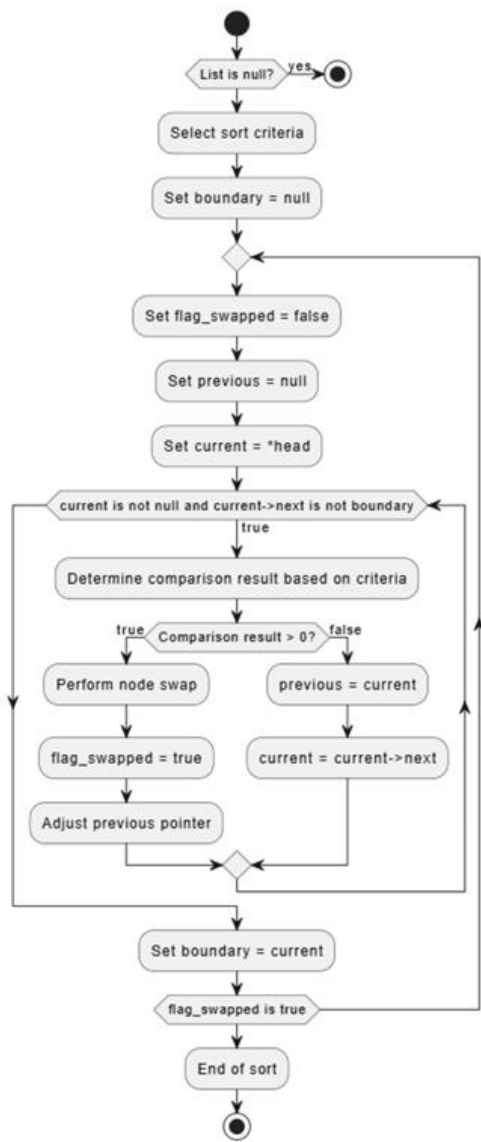


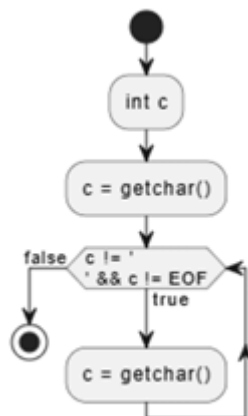
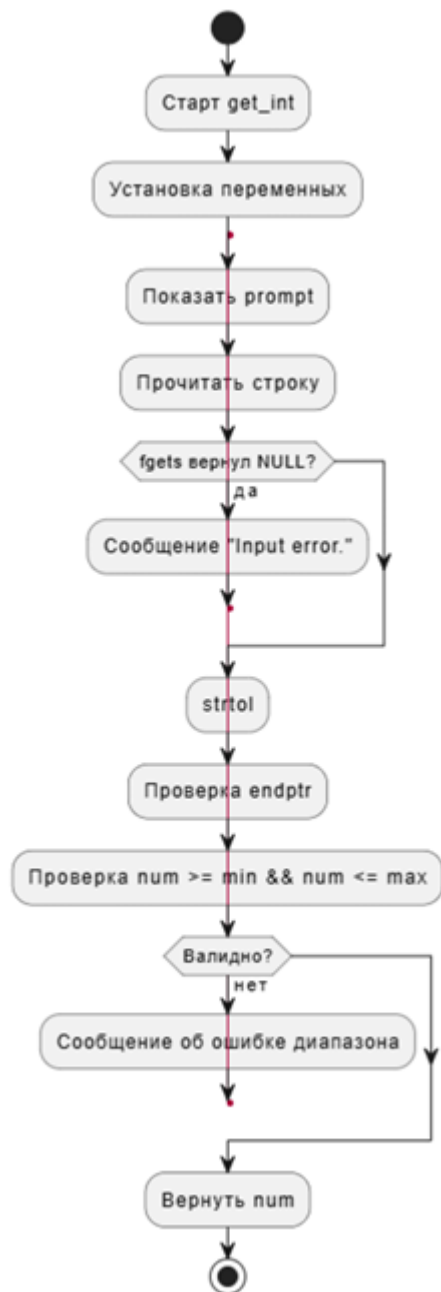


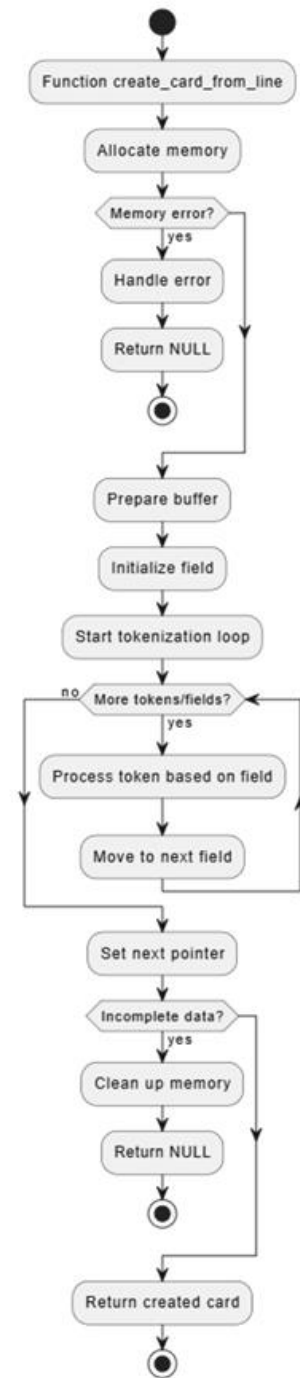
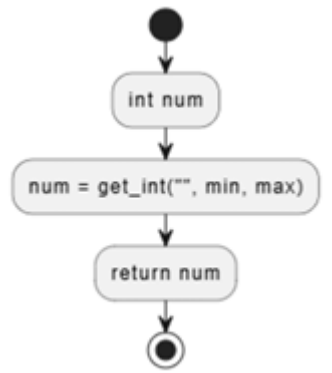












Контрольные примеры

№	Консоль																																																								
1	Electronic Card Index: Phones 0: Help 1: Add card 2: Edit card 3: Delete card 4: Print all cards 5: Search cards 6: Sort cards 7: Exit Select option (0-7): 1 Enter model: Tecno Spark 20 Enter brand: Tecno Enter RAM (GB): 8 Enter Storage (GB): 128 Enter Price: 14990 Enter Weight (g): 190 Card added. Press Enter to continue... Electronic Card Index: Phones 0: Help 1: Add card 2: Edit card 3: Delete card 4: Print all cards 5: Search cards 6: Sort cards 7: Exit Select option (0-7): 4 <table><tr><th>Idx</th><th>Model</th><th>Brand</th><th>RAM</th><th>Storage</th><th>Price</th><th>Weight</th></tr><tr><td>1</td><td>Apple iPhone 13</td><td>Apple</td><td>8</td><td>128</td><td>45663</td><td>167</td></tr><tr><td>2</td><td>Samsung Galaxy S22</td><td>Samsung</td><td>8</td><td>256</td><td>45663</td><td>178</td></tr><tr><td>3</td><td>Google Pixel 8</td><td>Google</td><td>12</td><td>128</td><td>27841</td><td>164</td></tr><tr><td>4</td><td>Xiaomi 13 Pro</td><td>Xiaomi</td><td>12</td><td>256</td><td>27841</td><td>184</td></tr><tr><td>5</td><td>OnePlus 11</td><td>OnePlus</td><td>12</td><td>256</td><td>23894</td><td>168</td></tr><tr><td>6</td><td>Sony Xperia 1 V</td><td>Sony</td><td>8</td><td>256</td><td>45694</td><td>201</td></tr><tr><td>7</td><td>Motorola Edge 40</td><td>Motorola</td><td>8</td><td>256</td><td>45722</td><td>188</td></tr></table>	Idx	Model	Brand	RAM	Storage	Price	Weight	1	Apple iPhone 13	Apple	8	128	45663	167	2	Samsung Galaxy S22	Samsung	8	256	45663	178	3	Google Pixel 8	Google	12	128	27841	164	4	Xiaomi 13 Pro	Xiaomi	12	256	27841	184	5	OnePlus 11	OnePlus	12	256	23894	168	6	Sony Xperia 1 V	Sony	8	256	45694	201	7	Motorola Edge 40	Motorola	8	256	45722	188
Idx	Model	Brand	RAM	Storage	Price	Weight																																																			
1	Apple iPhone 13	Apple	8	128	45663	167																																																			
2	Samsung Galaxy S22	Samsung	8	256	45663	178																																																			
3	Google Pixel 8	Google	12	128	27841	164																																																			
4	Xiaomi 13 Pro	Xiaomi	12	256	27841	184																																																			
5	OnePlus 11	OnePlus	12	256	23894	168																																																			
6	Sony Xperia 1 V	Sony	8	256	45694	201																																																			
7	Motorola Edge 40	Motorola	8	256	45722	188																																																			

	8 Huawei P60 Pro Huawei 16 128 26816 195 9 Asus ROG Phone 7 Asus 8 256 44348 169 10 Realme GT 3 Realme 12 128 45844 172 11 Tecno Spark 20 Tecno 8 128 14990 190 Press Enter to continue... Electronic Card Index: Phones 0: Help 1: Add card 2: Edit card 3: Delete card 4: Print all cards 5: Search cards 6: Sort cards 7: Exit Select option (0-7): 7 Bye!
2	Electronic Card Index: Phones 0: Help 1: Add card 2: Edit card 3: Delete card 4: Print all cards 5: Search cards 6: Sort cards 7: Exit Select option (0-7): 5 Search by: 1: Model 2: Brand 3: RAM 4: Storage 5: Price 6: Weight Select (1-6): 3 Enter value: 8 Matches: 1 Apple iPhone 13 Apple 8 128 45663 167 2 Samsung Galaxy S22 Samsung 8 256 45663 178 3 Sony Xperia 1 V Sony 8 256 45694 201 4 Motorola Edge 40 Motorola 8 256 45722 188 5 Asus ROG Phone 7 Asus 8 256 44348 169

	6 Tecno Spark 20 Tecno 8 128 14990 190 Press Enter to continue... Electronic Card Index: Phones 0: Help 1: Add card 2: Edit card 3: Delete card 4: Print all cards 5: Search cards 6: Sort cards 7: Exit Select option (0-7): 3 <table><tr><th>Idx</th><th>Model</th><th>Brand</th><th>RAM</th><th>Storage</th><th>Price</th><th>Weight</th></tr><tr><td>1</td><td>Apple iPhone 13</td><td>Apple</td><td>8</td><td>128</td><td>45663</td><td>167</td></tr><tr><td>2</td><td>Samsung Galaxy S22</td><td>Samsung</td><td>8</td><td>256</td><td>45663</td><td>178</td></tr><tr><td>3</td><td>Google Pixel 8</td><td>Google</td><td>12</td><td>128</td><td>27841</td><td>164</td></tr><tr><td>4</td><td>Xiaomi 13 Pro</td><td>Xiaomi</td><td>12</td><td>256</td><td>27841</td><td>184</td></tr><tr><td>5</td><td>OnePlus 11</td><td>OnePlus</td><td>12</td><td>256</td><td>23894</td><td>168</td></tr><tr><td>6</td><td>Sony Xperia 1 V</td><td>Sony</td><td>8</td><td>256</td><td>45694</td><td>201</td></tr><tr><td>7</td><td>Motorola Edge 40</td><td>Motorola</td><td>8</td><td>256</td><td>45722</td><td>188</td></tr><tr><td>8</td><td>Huawei P60 Pro</td><td>Huawei</td><td>16</td><td>128</td><td>26816</td><td>195</td></tr><tr><td>9</td><td>Asus ROG Phone 7</td><td>Asus</td><td>8</td><td>256</td><td>44348</td><td>169</td></tr><tr><td>10</td><td>Realme GT 3</td><td>Realme</td><td>12</td><td>128</td><td>45844</td><td>172</td></tr><tr><td>11</td><td>Tecno Spark 20</td><td>Tecno</td><td>8</td><td>128</td><td>14990</td><td>190</td></tr></table> Enter index to delete (0 - cancel): 11 Card deleted. Press Enter to continue...	Idx	Model	Brand	RAM	Storage	Price	Weight	1	Apple iPhone 13	Apple	8	128	45663	167	2	Samsung Galaxy S22	Samsung	8	256	45663	178	3	Google Pixel 8	Google	12	128	27841	164	4	Xiaomi 13 Pro	Xiaomi	12	256	27841	184	5	OnePlus 11	OnePlus	12	256	23894	168	6	Sony Xperia 1 V	Sony	8	256	45694	201	7	Motorola Edge 40	Motorola	8	256	45722	188	8	Huawei P60 Pro	Huawei	16	128	26816	195	9	Asus ROG Phone 7	Asus	8	256	44348	169	10	Realme GT 3	Realme	12	128	45844	172	11	Tecno Spark 20	Tecno	8	128	14990	190
Idx	Model	Brand	RAM	Storage	Price	Weight																																																																															
1	Apple iPhone 13	Apple	8	128	45663	167																																																																															
2	Samsung Galaxy S22	Samsung	8	256	45663	178																																																																															
3	Google Pixel 8	Google	12	128	27841	164																																																																															
4	Xiaomi 13 Pro	Xiaomi	12	256	27841	184																																																																															
5	OnePlus 11	OnePlus	12	256	23894	168																																																																															
6	Sony Xperia 1 V	Sony	8	256	45694	201																																																																															
7	Motorola Edge 40	Motorola	8	256	45722	188																																																																															
8	Huawei P60 Pro	Huawei	16	128	26816	195																																																																															
9	Asus ROG Phone 7	Asus	8	256	44348	169																																																																															
10	Realme GT 3	Realme	12	128	45844	172																																																																															
11	Tecno Spark 20	Tecno	8	128	14990	190																																																																															
3	Electronic Card Index: Phones 0: Help 1: Add card 2: Edit card 3: Delete card 4: Print all cards 5: Search cards 6: Sort cards 7: Exit Select option (0-7): 2 <table><tr><th>Idx</th><th>Model</th><th>Brand</th><th>RAM</th><th>Storage</th><th>Price</th><th>Weight</th></tr><tr><td>1</td><td>Apple iPhone 13</td><td>Apple</td><td>8</td><td>128</td><td>45663</td><td>167</td></tr><tr><td>2</td><td>Samsung Galaxy S22</td><td>Samsung</td><td>8</td><td>256</td><td>45663</td><td>178</td></tr></table>	Idx	Model	Brand	RAM	Storage	Price	Weight	1	Apple iPhone 13	Apple	8	128	45663	167	2	Samsung Galaxy S22	Samsung	8	256	45663	178																																																															
Idx	Model	Brand	RAM	Storage	Price	Weight																																																																															
1	Apple iPhone 13	Apple	8	128	45663	167																																																																															
2	Samsung Galaxy S22	Samsung	8	256	45663	178																																																																															

3	Google Pixel 8	Google	12	128 27841	164
4	Xiaomi 13 Pro	Xiaomi	12	256 27841	184
5	OnePlus 11	OnePlus	12	256 23894	168
6	Sony Xperia 1 V	Sony	8	256 45694	201
7	Motorola Edge 40	Motorola	8	256 45722	188
8	Huawei P60 Pro	Huawei	16	128 26816	195
9	Asus ROG Phone 7	Asus	8	256 44348	169
10	Realme GT 3	Realme	12	128 45844	172

Enter index to edit (0 - cancel): 3

Editing card #3:

Enter model [Google Pixel 8]:

Enter brand [Google]:

Enter RAM (GB) [12]: 8

Enter Storage (GB) [128]: 256

Enter Price [27841]:

Enter Weight (g) [164]: 170

Card updated.

Press Enter to continue...

Electronic Card Index: Phones

0: Help

1: Add card

2: Edit card

3: Delete card

4: Print all cards

5: Search cards

6: Sort cards

7: Exit

Select option (0-7): 6

Sort by:

1: Model

2: Brand

3: RAM

4: Storage

5: Price

6: Weight

Select (1-6): 6

Sorted.

Press Enter to continue...

Electronic Card Index: Phones

0: Help

1: Add card
2: Edit card
3: Delete card
4: Print all cards
5: Search cards
6: Sort cards
7: Exit
Select option (0-7): 4
Idx Model Brand RAM Storage Price Weight
1 Google Pixel 8 Google 8 256 27841 170
2 Apple iPhone 13 Apple 8 128 45663 167
3 OnePlus 11 OnePlus 12 256 23894 168
4 Asus ROG Phone 7 Asus 8 256 44348 169
5 Realme GT 3 Realme 12 128 45844 172
6 Xiaomi 13 Pro Xiaomi 12 256 27841 184
7 Samsung Galaxy S22 Samsung 8 256 45663 178
8 Motorola Edge 40 Motorola 8 256 45722 188
9 Huawei P60 Pro Huawei 16 128 26816 195
10 Sony Xperia 1 V Sony 8 256 45694 201
Press Enter to continue...

Текст программы

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_FIELD_LENGTH 64
#define MAX_LINE_LENGTH 256

#ifdef _WIN32
#define CLEAR_COMMAND "cls"
#else
#define CLEAR_COMMAND "clear"
#endif

```

```
/*  
Brand structure (doubly-linked list)
```

```
*/  
typedef struct Brand {  
    int id;  
    char *name;  
    struct Brand *prev;  
    struct Brand *next;  
} Brand;
```

```
/*  
Phone structure (singly-linked list)  
Each phone has a pointer to the Brand.
```

```
*/  
typedef struct Phone {  
    char model[MAX_FIELD_LENGTH];  
    Brand *brand;  
    int ram;  
    int storage;  
    int price;  
    int weight;  
    struct Phone *next;  
} Phone;
```

```
/* ----- Function prototypes ----- */  
void print_menu(void);  
void print_help(void);  
void pause_console(void);
```

void clear_screen(void);

Brand* add_brand(Brand **head, const char *name);

Brand* find_brand_by_name(Brand *head, const char *name);

void print_brands(Brand *head);

void free_brands(Brand *head, Phone *phone_head);

Phone* load_from_file(const char *filename, Brand **brands_head);

int save_to_file(const char *filename, Phone *phone_head);

Phone* add_card(Phone *phone_head, Brand **brands_head);

void print_cards(Phone *phone_head);

Phone* edit_card(Phone *phone_head, Brand **brands_head);

Phone* delete_card(Phone *phone_head, Brand **brands_head);

void search_cards(Phone *phone_head);

void sort_cards(Phone **phone_head);

void free_cards(Phone *phone_head);

int get_int(const char *prompt, int min, int max);

void flush_input(void);

int menu_choice(int min, int max);

**Phone* create_card_from_line(const char *line, Brand
brands_head);

void print_card(const Phone *card, int idx);

**void remove_brand_and_phones(Brand **brands_head, Phone
phone_head, int brand_id);

void remove_phone_by_brand(Phone **phone_head, int brand_id);

/* ----- Main program ----- */

int main(void)

```

{
    Brand *brands_head;
    Phone *phone_head;
    int running;
    int choice;
    char filename[] = "phones.csv";

    brands_head = NULL;
    phone_head = load_from_file(filename, &brands_head);
    if (!phone_head) {
        printf("Warning: Could not load data from '%s'. Starting with
empty card file.\n", filename);
    }

    running = 1;
    while (running) {
        clear_screen();
        print_menu();
        choice = menu_choice(0, 7);

        switch (choice) {
            case 0:
                print_help();
                pause_console();
                break;
            case 1:
                phone_head = add_card(phone_head, &brands_head);
                save_to_file(filename, phone_head);
                pause_console();

```

break;

case 2:

phone_head = edit_card(phone_head, &brands_head);

save_to_file(filename, phone_head);

pause_console();

break;

case 3:

phone_head = delete_card(phone_head, &brands_head);

save_to_file(filename, phone_head);

pause_console();

break;

case 4:

print_cards(phone_head);

pause_console();

break;

case 5:

search_cards(phone_head);

pause_console();

break;

case 6:

sort_cards(&phone_head);

save_to_file(filename, phone_head);

pause_console();

break;

case 7:

running = 0;

break;

default:

break;

```

    }
}

free_cards(phone_head);
free_brands(brands_head, NULL);
printf("Bye!\n");
return 0;
}

/* ----- Menu and UI ----- */
void print_menu(void)
{
    printf("Electronic Card Index: Phones\n");
    printf("0: Help\n");
    printf("1: Add card\n");
    printf("2: Edit card\n");
    printf("3: Delete card\n");
    printf("4: Print all cards\n");
    printf("5: Search cards\n");
    printf("6: Sort cards\n");
    printf("7: Exit\n");
    printf("Select option (0-7): ");
}

void print_help(void)
{
    printf("Help:\n");
    printf("This program manages an electronic card file (catalog) of  
phones.\n");
}

```

```

    printf("Data is read from 'phones.csv' and stored as two linked
lists in memory:\n");
    printf(" - list of brands (unique manufacturers)\n");
    printf(" - list of phones, each referencing a brand\n");
    printf("You can add, edit, delete, search, sort, and display
cards.\n");
    printf("If you delete a brand, all its phones are deleted!\n");
    printf("All changes are saved to 'phones.csv'.\n");
}

```

```

void pause_console(void)
{
    printf("Press Enter to continue...");
    fflush(stdout);
    while(getchar() != '\n');
}

```

```

void clear_screen(void)
{
    system(CLEAR_COMMAND);
}

```

```

/* ----- Brand list functions ----- */

```

```

/* Add (if necessary) and return pointer to unique brand */

```

```

Brand* add_brand(Brand **head, const char *name)
{
    Brand *find, *new_brand, *tail;
    int id;

```



```

if (!name || !*name) return NULL;
find = find_brand_by_name(*head, name);
if (find) return find;

/* create new */
new_brand = (Brand*)malloc(sizeof(Brand));
if (!new_brand) {
    printf("Memory error.\n");
    return NULL;
}

/* assign id: max+1 or 1 */
id = 1;
tail = *head;
while (tail && tail->next) tail = tail->next;
if (tail) id = tail->id + 1;
new_brand->id = id;
new_brand->name = (char*)malloc(strlen(name) + 1);
if (!new_brand->name) {
    free(new_brand);
    printf("Memory error.\n");
    return NULL;
}
strcpy(new_brand->name, name);
new_brand->prev = tail;
new_brand->next = NULL;
if (tail) tail->next = new_brand;
else *head = new_brand;
return new_brand;
}

```

/* Find brand by name */

Brand* find_brand_by_name(Brand *head, const char *name)

```
{  
    while (head) {  
        if (strcmp(head->name, name) == 0) return head;  
        head = head->next;  
    }  
    return NULL;  
}
```

/* Print list of brands */

void print_brands(Brand *head)

```
{  
    printf("Brand List:\n");  
    while (head) {  
        printf(" %d: %s\n", head->id, head->name);  
        head = head->next;  
    }  
}
```

/* Free brands (and optionally delete all phones referencing them) */

void free_brands(Brand *head, Phone *phone_head)

```
{  
    Brand *tmp;  
    while (head) {  
        tmp = head;  
        head = head->next;  
        if (tmp->name) free(tmp->name);  
    }
```

```

        free(tmp);
    }
}

/* Remove a brand by id and all phones of that brand */
void remove_brand_and_phones(Brand **brands_head, Phone
**phone_head, int brand_id)
{
    Brand *cur, *prev;
    if (!brands_head || !*brands_head) return;
    cur = *brands_head;
    prev = NULL;
    while (cur) {
        if (cur->id == brand_id) {
            /* Remove from list */
            if (prev) prev->next = cur->next;
            else *brands_head = cur->next;
            if (cur->next) cur->next->prev = prev;
            /* Remove all phones of this brand */
            remove_phone_by_brand(phone_head, brand_id);
            if (cur->name) free(cur->name);
            free(cur);
            printf("Brand and all its phones deleted.\n");
            return;
        }
        prev = cur;
        cur = cur->next;
    }
}

```

```

/* Remove phones by brand id */
void remove_phone_by_brand(Phone **phone_head, int brand_id)
{
    Phone *cur, *prev, *tmp;
    cur = *phone_head;
    prev = NULL;
    while (cur) {
        if (cur->brand && cur->brand->id == brand_id) {
            tmp = cur;
            if (prev) prev->next = cur->next;
            else *phone_head = cur->next;
            cur = cur->next;
            free(tmp);
        } else {
            prev = cur;
            cur = cur->next;
        }
    }
}

/* ----- Phone list functions ----- */

/* Load cards & brands from CSV file */
Phone* load_from_file(const char *filename, Brand **brands_head)
{
    FILE *f;
    char line[MAX_LINE_LENGTH];
    Phone *phone_head, *tail, *card;

```

```

if (!brands_head) return NULL;
*brands_head = NULL;

f = fopen(filename, "r");
if (!f) return NULL;
phone_head = NULL;
tail = NULL;

/* Skip header */
if (!fgets(line, sizeof(line), f)) {
    fclose(f);
    return NULL;
}

while (fgets(line, sizeof(line), f)) {
    card = create_card_from_line(line, brands_head);
    if (!card) continue;
    if (!phone_head) phone_head = card;
    else tail->next = card;
    tail = card;
}
fclose(f);
return phone_head;
}

/* Save cards to CSV */
int save_to_file(const char *filename, Phone *phone_head)
{
    FILE *f;

```

```

Phone *p;
f = fopen(filename, "w");
if (!f) {
    printf("Error: cannot open '%s' for writing.\n", filename);
    return 0;
}
fprintf(f, "Model,Brand,RAM,Storage,Price,Weight\n");
for (p = phone_head; p; p = p->next) {
    fprintf(f, "%s,%s,%d,%d,%d,%d\n", p->model, p->brand ? p-
>brand->name : "", p->ram, p->storage, p->price, p->weight);
}
fclose(f);
return 1;
}

```

/* Parse CSV line into Phone struct */

Phone* create_card_from_line(const char *line, Brand

****brands_head)**

{

Phone *card;

char buf[MAX_LINE_LENGTH];

char *token;

int field;

char field_model[MAX_FIELD_LENGTH],

field_brand[MAX_FIELD_LENGTH];

int field_ram, field_storage, field_price, field_weight;

Brand *brand_ptr;

if (!line || !brands_head) return NULL;

```

strncpy(buf, line, sizeof(buf));
buf[sizeof(buf) - 1] = '\0';
field = 0;
field_model[0] = field_brand[0] = '\0';
field_ram = field_storage = field_price = field_weight = 0;

token = strtok(buf, ",\n");
while (token && field < 6) {
    switch (field) {
        case 0: strncpy(field_model, token, MAX_FIELD_LENGTH-
1); field_model[MAX_FIELD_LENGTH-1]='\0'; break;
        case 1: strncpy(field_brand, token, MAX_FIELD_LENGTH-
1); field_brand[MAX_FIELD_LENGTH-1]='\0'; break;
        case 2: field_ram = atoi(token); break;
        case 3: field_storage = atoi(token); break;
        case 4: field_price = atoi(token); break;
        case 5: field_weight = atoi(token); break;
        default: break;
    }
    field++;
    token = strtok(NULL, ",\n");
}
if (field < 6) return NULL;

card = (Phone*)malloc(sizeof(Phone));
if (!card) {
    printf("Memory allocation failed.\n");
    return NULL;
}

```

```

    strncpy(card->model, field_model, MAX_FIELD_LENGTH-1);
    card->model[MAX_FIELD_LENGTH-1] = '\0';
    brand_ptr = add_brand(brands_head, field_brand);
    card->brand = brand_ptr;
    card->ram = field_ram;
    card->storage = field_storage;
    card->price = field_price;
    card->weight = field_weight;
    card->next = NULL;
    return card;
}

```

/ Add new card */*

```

Phone* add_card(Phone *phone_head, Brand **brands_head)

```

```

{
    Phone *card, *tail;
    char buf[MAX_FIELD_LENGTH];
    Brand *brand_ptr;
    int i, brand_choice;
    Brand *bcur;
    int count;

    card = (Phone*)malloc(sizeof(Phone));
    if (!card) {
        printf("Memory allocation failed.\n");
        return phone_head;
    }
}

```

```

printf("Enter model: ");

```



```

if (!fgets(buf, sizeof(buf), stdin)) { free(card); return phone_head; }
buf[strcspn(buf, "\n")] = '\0';
strncpy(card->model, buf, MAX_FIELD_LENGTH-1);
card->model[MAX_FIELD_LENGTH-1] = '\0';

/* Choose existing brand or add new */
printf("Existing brands:\n");
bcur = *brands_head;
count = 0;
while (bcur) {
    printf(" %d: %s\n", bcur->id, bcur->name);
    count++;
    bcur = bcur->next;
}
printf("0: Add new brand\n");
brand_choice = get_int("Choose brand by id (or 0 to add): ", 0,
(count>0)?1000:0);

if (brand_choice == 0) {
    printf("Enter new brand name: ");
    if (!fgets(buf, sizeof(buf), stdin)) { free(card); return
phone_head; }
    buf[strcspn(buf, "\n")] = '\0';
    brand_ptr = add_brand(brands_head, buf);
} else {
    bcur = *brands_head;
    while (bcur && bcur->id != brand_choice) bcur = bcur->next;
    if (bcur) brand_ptr = bcur;
    else {

```

```

        printf("Invalid brand id.\n");
        free(card);
        return phone_head;
    }
}

card->brand = brand_ptr;

card->ram = get_int("Enter RAM (GB): ", 1, 1024);
card->storage = get_int("Enter Storage (GB): ", 1, 8192);
card->price = get_int("Enter Price: ", 1, 1000000);
card->weight = get_int("Enter Weight (g): ", 1, 10000);

card->next = NULL;
if (!phone_head) return card;
tail = phone_head;
while (tail->next) tail = tail->next;
tail->next = card;
return phone_head;
}

/* Print all cards */
void print_cards(Phone *phone_head)
{
    Phone *p;
    int idx;
    if (!phone_head) {
        printf("No cards.\n");
        return;
    }
}

```

```

    printf("Idx | %-20s | %-12s | %3s | %7s | %6s | %6s\n", "Model",
"Brand", "RAM", "Storage", "Price", "Weight");
    idx = 1;
    for (p = phone_head; p; p = p->next) {
        print_card(p, idx);
        idx++;
    }
}

```

/* Print single card */

```

void print_card(const Phone *card, int idx)
{
    printf("%3d | %-20s | %-12s | %3d | %7d | %6d | %6d\n", idx,
card->model, card->brand ? card->brand->name : "", card->ram,
card->storage, card->price, card->weight);
}

```

/* Edit a card by index */

```

Phone* edit_card(Phone *phone_head, Brand **brands_head)
{
    int idx, i, brand_choice, count;
    Phone *p;
    char buf[MAX_FIELD_LENGTH];
    Brand *bcur, *brand_ptr;
    if (!phone_head) {
        printf("No cards to edit.\n");
        return phone_head;
    }
    print_cards(phone_head);
}

```

```

idx = get_int("Enter index to edit (0 - cancel): ", 0, 1000);
if (idx == 0) return phone_head;
p = phone_head;
for (i = 1; p && i < idx; i++) p = p->next;
if (!p) {
    printf("Invalid index.\n");
    return phone_head;
}
printf("Editing card #%d:\n", idx);

printf("Enter model [%s]: ", p->model);
if (fgets(buf, sizeof(buf), stdin)) {
    buf[strcspn(buf, "\n")] = '\0';
    if (buf[0]) strncpy(p->model, buf, MAX_FIELD_LENGTH-1);
}

printf("Existing brands:\n");
bcur = *brands_head; count = 0;
while (bcur) { printf("%d: %s\n", bcur->id, bcur->name);
count++; bcur = bcur->next; }
printf("0: Add new brand\n");
brand_choice = get_int("Choose brand by id (or 0 to add): ", 0,
(count>0)?1000:0);
if (brand_choice == 0) {
    printf("Enter new brand name: ");
    if (fgets(buf, sizeof(buf), stdin)) {
        buf[strcspn(buf, "\n")] = '\0';
        brand_ptr = add_brand(brands_head, buf);
        p->brand = brand_ptr;
    }
}

```

```
    }  
} else {  
    bcur = *brands_head;  
    while (bcur && bcur->id != brand_choice) bcur = bcur->next;  
    if (bcur) p->brand = bcur;  
}
```

```
printf("Enter RAM (GB) [%d]: ", p->ram);  
if (fgets(buf, sizeof(buf), stdin)) {  
    int val;  
    buf[strcspn(buf, "\n")] = '\0';  
    if (buf[0]) {  
        val = atoi(buf);  
        if (val > 0) p->ram = val;  
    }  
}  
printf("Enter Storage (GB) [%d]: ", p->storage);  
if (fgets(buf, sizeof(buf), stdin)) {  
    int val;  
    buf[strcspn(buf, "\n")] = '\0';  
    if (buf[0]) {  
        val = atoi(buf);  
        if (val > 0) p->storage = val;  
    }  
}  
printf("Enter Price [%d]: ", p->price);  
if (fgets(buf, sizeof(buf), stdin)) {  
    int val;  
    buf[strcspn(buf, "\n")] = '\0';
```

```

        if (buf[0]) {
            val = atoi(buf);
            if (val > 0) p->price = val;
        }
    }

    printf("Enter Weight (g) [%d]: ", p->weight);
    if (fgets(buf, sizeof(buf), stdin)) {
        int val;
        buf[strcspn(buf, "\n")] = '\0';
        if (buf[0]) {
            val = atoi(buf);
            if (val > 0) p->weight = val;
        }
    }

    printf("Card updated.\n");
    return phone_head;
}

/* Delete a card or a brand (and all its phones) */
Phone* delete_card(Phone *phone_head, Brand **brands_head)
{
    int choice, idx, i, brand_id;
    Phone *p, *prev, *tmp;
    Brand *bcur;
    if (!phone_head) {
        printf("No cards to delete.\n");
        return phone_head;
    }
    printf("Delete:\n1: a card\n2: a brand and all its phones\n");

```

```

choice = get_int("Choose (1-2): ", 1, 2);
if (choice == 1) {
    print_cards(phone_head);
    idx = get_int("Enter index to delete (0 - cancel): ", 0, 1000);
    if (idx == 0) return phone_head;
    p = phone_head;
    prev = NULL;
    for (i = 1; p && i < idx; i++) {
        prev = p;
        p = p->next;
    }
    if (!p) {
        printf("Invalid index.\n");
        return phone_head;
    }
    if (!prev) phone_head = p->next;
    else prev->next = p->next;
    free(p);
    printf("Card deleted.\n");
    return phone_head;
} else {
    print_brands(*brands_head);
    brand_id = get_int("Enter brand id to delete (0 - cancel): ", 0,
1000);
    if (brand_id == 0) return phone_head;
    remove_brand_and_phones(brands_head, &phone_head,
brand_id);
    return phone_head;
}

```

```
}
```

```
/* Search cards by parameter */
```

```
void search_cards(Phone *phone_head)
```

```
{
```

```
    int param;
```

```
    char buf[MAX_FIELD_LENGTH];
```

```
    int val;
```

```
    Phone *p;
```

```
    int found;
```

```
    printf("Search by:\n");
```

```
    printf("1: Model\n2: Brand\n3: RAM\n4: Storage\n5: Price\n6:
```

```
Weight\n");
```

```
    param = get_int("Select (1-6): ", 1, 6);
```

```
    found = 0;
```

```
    if (param == 1 || param == 2) {
```

```
        printf("Enter string: ");
```

```
        if (!fgets(buf, sizeof(buf), stdin)) return;
```

```
        buf[strcspn(buf, "\n")] = '\0';
```

```
        for (p = phone_head; p; p = p->next) {
```

```
            if ((param == 1 && strstr(p->model, buf)) ||
```

```
                (param == 2 && p->brand && strstr(p->brand->name,
```

```
buf))) {
```

```
                if (!found) printf("Matches:\n");
```

```
                print_card(p, found+1);
```

```
                found++;
```

```
            }
```

```
        }
```



```

} else {
    val = get_int("Enter value: ", 0, 1000000);
    for (p = phone_head; p; p = p->next) {
        if ((param == 3 && p->ram == val) ||
            (param == 4 && p->storage == val) ||
            (param == 5 && p->price == val) ||
            (param == 6 && p->weight == val)) {
            if (!found) printf("Matches:\n");
            print_card(p, found+1);
            found++;
        }
    }
}

if (!found) {
    printf("No matches found.\n");
}
}

/* Sort cards by parameter (bubble sort) */
void sort_cards(Phone **phone_head)
{
    int param, swapped;
    Phone *p, *prev, *temp, *lptr;
    if (!*phone_head) {
        printf("No cards to sort.\n");
        return;
    }

    printf("Sort by:\n1: Model\n2: Brand\n3: RAM\n4: Storage\n5:
Price\n6: Weight\n");

```

```

param = get_int("Select (1-6): ", 1, 6);
lptr = NULL;
do {
    swapped = 0;
    prev = NULL;
    p = *phone_head;
    while (p && p->next != lptr) {
        int cmp = 0;
        switch (param) {
            case 1: cmp = strcmp(p->model, p->next->model); break;
            case 2: cmp = strcmp(p->brand ? p->brand->name : "", p-
>next->brand ? p->next->brand->name : ""); break;
            case 3: cmp = p->ram - p->next->ram; break;
            case 4: cmp = p->storage - p->next->storage; break;
            case 5: cmp = p->price - p->next->price; break;
            case 6: cmp = p->weight - p->next->weight; break;
        }
        if (cmp > 0) {
            if (prev) prev->next = p->next;
            else *phone_head = p->next;
            temp = p->next->next;
            p->next->next = p;
            p->next = temp;
            swapped = 1;
            prev = (*phone_head == p) ? p : prev->next;
        } else {
            prev = p;
            p = p->next;
        }
    }
}

```

```

    }
    lptr = p;
} while (swapped);
printf("Sorted.\n");
}

```

```

/* Free all allocated cards */
void free_cards(Phone *phone_head)
{
    Phone *p;
    while (phone_head) {
        p = phone_head;
        phone_head = phone_head->next;
        free(p);
    }
}

```

```

/* ----- Utility functions ----- */

```

```

/* Get integer input with prompt and range */
int get_int(const char *prompt, int min, int max)
{
    int num;
    char buf[MAX_LINE_LENGTH];
    char *endptr;
    int valid;

    do {
        printf(" %s", prompt);

```

```

    if (!fgets(buf, sizeof(buf), stdin)) {
        printf("Input error.\n");
        continue;
    }
    num = (int)strtol(buf, &endptr, 10);
    valid = (endptr != buf && (*endptr == '\n' || *endptr == '\0')
&& num >= min && num <= max);
    if (!valid) {
        printf("Please enter integer between %d and %d.\n", min,
max);
    }
} while (!valid);
return num;
}

```

```

void flush_input(void)
{
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}

```

```

int menu_choice(int min, int max)
{
    int num;
    num = get_int("", min, max);
    return num;
}

```

Примеры выполнения программы

Пример 1:

Electronic Card Index: Phones

0: Help

1: Add card

2: Edit card

3: Delete card

4: Print all cards

5: Search cards

6: Sort cards

7: Exit

Select option (0-7): 1

Enter model: Nokia 3310

Enter brand: Nokia

Enter RAM (GB): 1

Enter Storage (GB): 16

Enter Price: 5000

Enter Weight (g): 150

Press Enter to continue...

Electronic Card Index: Phones

0: Help

1: Add card

2: Edit card

3: Delete card

4: Print all cards

5: Search cards

6: Sort cards

7: Exit

Select option (0-7): 5

Search by:

1: Model

2: Brand

3: RAM

4: Storage

5: Price

6: Weight

Select (1-6): 2

Enter string: Nokia

Matches:

1		Nokia 3310		Nokia		1		16		5000		150
---	--	------------	--	-------	--	---	--	----	--	------	--	-----

Press Enter to continue...

Пример 2:

Electronic Card Index: Phones

0: Help

1: Add card

2: Edit card

3: Delete card

4: Print all cards

5: Search cards

6: Sort cards

7: Exit

Select option (0-7): 4

Idx	Model	Brand	RAM	Storage	Price	Weight
1	Apple iPhone 13	Apple	8	128	45663	167
2	Samsung Galaxy S22	Samsung	8	256	45663	178
3	Google Pixel 8	Google	8	256	27841	170
4	Xiaomi 13 Pro	Xiaomi	12	256	27841	184
5	OnePlus 11	OnePlus	12	256	23894	168

Press Enter to continue...

Electronic Card Index: Phones

0: Help

1: Add card

2: Edit card

3: Delete card

4: Print all cards

5: Search cards

6: Sort cards

7: Exit

Select option (0-7): 3

Idx	Model	Brand	RAM	Storage	Price	Weight
1	Apple iPhone 13	Apple	8	128	45663	167
2	Samsung Galaxy S22	Samsung	8	256	45663	178
3	Google Pixel 8	Google	8	256	27841	170
4	Xiaomi 13 Pro	Xiaomi	12	256	27841	184
5	OnePlus 11	OnePlus	12	256	23894	168

Enter index to delete (0 - cancel): 2

Card deleted.

Press Enter to continue...

Electronic Card Index: Phones

0: Help

1: Add card

2: Edit card

3: Delete card

4: Print all cards

5: Search cards

6: Sort cards

7: Exit

Select option (0-7): 4

Idx	Model	Brand	RAM	Storage	Price	Weight
1	Apple iPhone 13	Apple	8	128	45663	167
2	Google Pixel 8	Google	8	256	27841	170
3	Xiaomi 13 Pro	Xiaomi	12	256	27841	184
4	OnePlus 11	OnePlus	12	256	23894	168

Press Enter to continue...

Заключение

В процессе выполнения курсовой работы была полностью реализована электронная картотека мобильных телефонов.

Использованные стандартные заголовочные файлы и функции:

1. <stdio.h>

Функции: printf(), fgets(), fopen(), fclose(), fprintf(), fscanf()

Предоставляют функционал для текстового ввода/вывода, как с клавиатуры, так и с файла. Используются для организации интерфейса пользователя и работы с CSV-файлом.

2. <stdlib.h>

Функции: malloc(), free(), atoi(), strtol(), system()

Реализуют управление динамической памятью (добавление, удаление карточек), обработку ошибок, выполнение системной команды очистки экрана и преобразование строк в числа.

3. <string.h>

Функции: strncpy(), strtok(), strstr(), strcmp(), strlen()

Используются для обработки строк: копирование, поиск подстрок, токенизация строк CSV при загрузке, сравнение при сортировке и поиске карточек.

4. <windows.h> / <unistd.h>

Задействованы через директиву #ifdef _WIN32 для реализации кроссплатформенной очистки экрана (system("cls") для Windows, system("clear") для Linux/macOS).

Выполненные требования

Структуры данных: реализована структура Phone, объединенная в односвязный список через указатель next.

Динамическая память: все карточки создаются и удаляются с использованием malloc() / free(); предусмотрены проверки успешности выделения памяти.

Модульность: каждая функция выполняет строго определённую задачу. Передача данных между функциями реализована через параметры.

Меню: реализовано главное меню с вложенной логикой обработки пунктов и повторением работы программы до выхода.

Файлы: реализован ввод/вывод с диска. Программа читает CSV-файл при запуске, и сохраняет изменения после каждого действия.

Поиск и сортировка: реализованы по шести параметрам: model, brand, ram, storage, price, weight.

Интерфейс: простой и удобный текстовый интерфейс с приглашениями и сообщениями, диалоговые окна, ожидание ввода.

Оформление кода: соблюден стиль C90. Все переменные описаны в начале функций. Комментарии — на английском. Прототипы функций выделены в отдельный блок. Реализация полностью следует требованиям strict C90.

Вывод

Программа продемонстрировала все этапы обработки данных:

Чтение из файла;

Диалоговый ввод от пользователя;

Хранение информации в динамической структуре;

Модификация, удаление, добавление;

Поиск и сортировка;

Вывод на экран и сохранение результата в файл.

Также реализована очистка экрана в зависимости от операционной системы, обработка ошибок (файлы, память, ввод), повторение команд в цикле, вывод помощи.